SECTION-A

Q1. Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

Ans: I would recommend the **Agile methodology** because the project has frequently changing requirements and the client wants to review working features every two weeks. Agile works in short iterations called sprints, allowing the team to receive feedback and make changes regularly.

If the wrong methodology is chosen, two risks are:

1. **Changing requirements may become difficult and expensive** to handle in Waterfall.
2. **Client feedback may come too late**, causing the team to build features that do not meet the client's expectations.

Q2. Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

Ans: The team should avoid pushing incomplete code directly to the main branch. Each developer should create a separate feature branch, make changes there, commit the changes, and then create a Pull Request for review before merging into main.

Two important Git commands are:

- **git checkout -b feature-name — creates and switches to a new branch for developing a feature.**
- **git add . — stages the changed files before committing.**
- **git commit -m "message" — saves the changes with a meaningful message.**
- The appropriate HTML input types are:

Field	Input Type
Applicant name	`text`
Email address	`email`
Phone number	`tel`
Preferred domain	`select`
Resume upload	`file`

- To make fields compulsory, we can use the **`required`** attribute.
- For email validation, we can use:

- `<input type="email" required>`
- The `email` type checks whether the entered value follows a basic email format, while `required` prevents the field from being left empty.
- However, HTML5 validation alone is **not sufficient for production-level validation** because users can bypass browser-side validation, and client-side validation cannot be trusted for security. Server-side validation is also required to properly validate and sanitize submitted data.
-

The team should also review and test the Pull Request before merging it into main. This prevents incomplete or broken code from directly affecting the main branch.

Q4. Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.

Ans: Bootstrap's grid system divides the page into **12 columns**. We can use responsive classes to specify how many columns an article should occupy at different screen sizes.

Explanation:

- col-12 → On small/mobile screens, the article takes all **12 columns**, so there is **1 article per row**.
- col-md-4 → On medium and larger screens, the article takes **4 of 12 columns**, allowing **3 articles per row**.

Therefore, Bootstrap automatically creates the required responsive layout without writing custom media queries.

Q5. Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

Ans: An **array** is suitable because all 30 rainfall readings are of the same type and can be stored under one variable name. We can access each reading using an index and easily process them using loops.

Using 30 separate variables would make the program longer, harder to manage, and harder to process.

The program should follow two steps:

Step 1 — Calculate the average:
Use a loop to add all 30 rainfall values and then divide the total by 30.

Step 2 — Find above-average days:
Use another loop to compare each day's rainfall with the calculated average and print the days whose rainfall is greater than the average.

Both steps cannot normally be completed correctly in a single straightforward loop because we need to know the **final average first** before we can determine which individual values are above that average.

Q6. Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Ans: The program can crash because an empty string contains no actual characters. If the pointer is dereferenced without first checking whether it points to a valid character, the program may access invalid memory or continue incorrectly.

Before dereferencing the pointer, the program should check that the pointer is **not NULL** and, while traversing a string, check whether the current character is the string terminator '\0'.

For example, conceptually:

pointer != NULL

AND

*pointer != '\0'

Pointer traversal moves through the string by changing the pointer, for example from one character's address to the next. Array indexing instead accesses characters using positions such as name[0], name[1], etc.

For a beginner, **array indexing is generally easier to understand and debug**, because the positions are explicit. However, both approaches can safely handle empty strings if the correct checks are performed.